

quantum-group: Exact symbolic verification of R-matrix and Yang–Baxter identities for $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$

Taha Berk Terekli*

Department of Mathematics, Yıldız Technical University, Istanbul, Türkiye

* terekli@tahaberk.com, ORCID: [0009-0004-8266-1116](https://orcid.org/0009-0004-8266-1116)

Abstract

Explicit R-matrices of quantum groups are routinely transcribed from the literature, yet whether they satisfy an identity depends on basis order, tensor placement, the coproduct and, for superalgebras, the Koszul sign rule. `quantum-group` is a Python package that builds such matrices exactly with SymPy and returns every identity as an exact residual matrix, whose nonzero entries locate the convention that disagrees. It covers $U_q(\mathfrak{sl}_2)$ modules, tensor products and the fundamental R-matrix, and the graded Yang–Baxter equation for $GL_q(2|1)$. Applied to the thesis it was written for, it exposed an R-matrix that satisfies the Yang–Baxter equation but intertwines the opposite coproduct.

Copyright attribution to authors.

This work is a submission to SciPost Physics Codebases.

License information to appear upon publication.

Publication information to appear upon publication.

Received Date

Accepted Date

Published Date

1

2

Contents

1	Introduction	2
2	Mathematical background	3
3	Design and workings	4
4	Installation and usage	7
5	Worked examples	9
6	Testing and benchmarks	12
7	Scope, limitations and extensions	15
8	Conclusion	16
A	Reference of the public interface	17
References		19

14

15

1 Introduction

Quantum groups are one-parameter deformations of classical symmetry algebras. Their representations yield solutions of the quantum Yang–Baxter equation (QYBE), which is why they appear in exactly solvable models of statistical mechanics, in knot invariants and in representation theory [1–4]. Much of the working knowledge in this area is recorded as explicit matrices whose entries are rational functions of the deformation parameter q : an R-matrix on $V \otimes V$, the generators of a module, a change of basis to irreducible summands. Before such a matrix can be reused it has to be checked, and checking it means multiplying 9×9 , 27×27 or 81×81 matrices whose meaning depends on conventions that texts state in different ways, or not at all.

The characteristic mistakes are not arithmetic ones. A matrix is transcribed in a different basis order from the one its source used. R_{13} is formed by conjugating with the wrong permutation. An R-matrix is paired with the opposite of the coproduct it actually intertwines; this still satisfies the QYBE, so the most common test does not detect it. In a $\mathbb{Z}_2$ -graded setting an ordinary flip is used where the Koszul sign rule requires a super-permutation. And a check that is run at a floating-point value of q can hide cancellations or produce spurious nonzero entries. Each of these errors produces output that looks plausible.

quantum-group [5] addresses this with a deliberately narrow design. Every object is an ordinary exact sympy.Matrix [6]; conventions (basis order, coproduct, parities) are fixed in documented functions; and every identity is exposed as a *residual*, the exact matrix LHS–RHS, which is then tested entry by entry. For $U_q(\mathfrak{sl}_2)$ the package constructs the irreducible modules V_n , checks the defining relations and generator-level Hopf identities, builds tensor products through the coproduct, computes Clebsch–Gordan highest-weight vectors, and verifies the fundamental R-matrix: QYBE, braid and Hecke relations and coproduct intertwining. For the quantum supergroup $GL_q(2|1)$ introduced by Çelik and Çelik [7], whose third basis vector is odd, it constructs the published 9×9 R-matrix and verifies the graded Yang–Baxter equation on three factors and on every triple of factors in $V^{\otimes 4}$.

The software was developed for and used in the author’s undergraduate thesis [8], where it produced the thesis’s tables, matrix figures and Yang–Baxter verifications. After the thesis was submitted in June 2026, the software was used to audit it: an explicit intertwining residual showed that the thesis’s R-matrix intertwines the opposite of the coproduct used in the text, an error that passing QYBE, braid and Hecke checks had not revealed (Sec. 5.2). The correction is recorded in the repository (thesis/ERRATA.md, AUDIT.md), and the check is now part of the API and the test suite. No new mathematical results are claimed; the contribution is the software and its verification method.

The intended users are researchers who need to confirm a matrix identity from the literature in their own conventions: mathematical physicists working with R-matrices and integrable models, representation theorists and computer-algebra users who want an inspectable reference calculation, and students reproducing standard quantum-group computations.

Related software. GAP’s QuaGroup package [9–11] and SageMath [12], which provides a QuantumGroup interface to it, work with the quantized enveloping algebra $U_q(\mathfrak{g})$ of any semisimple Lie algebra $\mathfrak{g}$: they construct PBW-type and canonical bases, highest-weight modules and their R-matrices; SageMath also implements crystals for the quantum superalgebras $U_q(\mathfrak{gl}(m|n))$ [13]. These systems are far broader, and for general $U_q(\mathfrak{g})$ work they are the appropriate tools. quantum-group does not replace them. Its added value is that of a small verification layer for concrete matrix calculations: it keeps the basis order and coproduct of a particular text, returns the matrices and residuals themselves rather than only a verdict, runs in an ordinary Python environment, encodes the graded sign rule once for every tensor place-

ment, and includes a graded $GL_q(2|1)$ example that lies outside QuaGroup's documented $U_q(\mathfrak{g})$ scope. The two are complementary: Sec. 6.3 uses quantum-group to test matrices computed by QuaGroup and to identify the conventions in which the systems differ.

Outline. Section 2 collects the definitions the package implements, and Sec. 3 describes how it works and why it is built this way. Section 4 explains installation and the public interface, Sec. 5 gives worked examples, including the thesis audit, and Sec. 6 documents testing, a cross-check against GAP's QuaGroup and performance. Section 7 states the scope and the ways the code can be reused or extended. Appendix A is a reference of the public interface.

2 Mathematical background

This section collects the definitions that the package implements, in the conventions it uses. Standard references are Refs. [1–4]; the graded case follows Ref. [7].

2.1 The algebra $U_q(\mathfrak{sl}_2)$ and its modules

Let q be an indeterminate, or a nonzero complex number that is not a root of unity. $U_q(\mathfrak{sl}_2)$ is the algebra generated by E, F, K and K^{-1} with the relations

$$\begin{aligned} \text{(R1)} \quad KK^{-1} &= K^{-1}K = 1, & \text{(R2)} \quad KEK^{-1} &= q^2E, \\ \text{(R3)} \quad KFK^{-1} &= q^{-2}F, & \text{(R4)} \quad EF - FE &= \frac{K - K^{-1}}{q - q^{-1}}. \end{aligned} \quad (1)$$

It is a Hopf algebra with coproduct, counit and antipode

$$\begin{aligned} \Delta(E) &= E \otimes 1 + K \otimes E, & \Delta(F) &= F \otimes K^{-1} + 1 \otimes F, \\ \Delta(K^{\pm 1}) &= K^{\pm 1} \otimes K^{\pm 1}, \end{aligned} \quad (2)$$

$$\begin{aligned} \varepsilon(E) &= \varepsilon(F) = 0, & \varepsilon(K^{\pm 1}) &= 1, \\ S(E) &= -K^{-1}E, & S(F) &= -FK, & S(K^{\pm 1}) &= K^{\mp 1}. \end{aligned} \quad (3)$$

The opposite coproduct is $\Delta^{\text{op}} = P \circ \Delta$, where P exchanges the two tensor factors; on a tensor product of modules, $\Delta^{\text{op}}(X) = P \Delta(X) P$. With the q -integers

$$[k] = \frac{q^k - q^{-k}}{q - q^{-1}} = q^{k-1} + q^{k-3} + \cdots + q^{1-k}, \quad [k]! = [1][2] \cdots [k], \quad (4)$$

the irreducible module V_n of dimension $n + 1$ has a basis $v_0, \dots, v_n$ on which

$$\begin{aligned} Kv_k &= q^{n-2k} v_k, & Fv_k &= v_{k+1} \quad (Fv_n = 0), \\ Ev_k &= [k][n-k+1] v_{k-1} \quad (Ev_0 = 0). \end{aligned} \quad (5)$$

A tensor product $V \otimes W$ of modules is a module through Δ . For generic q the q -analogue of the Clebsch–Gordan rule holds, $V_m \otimes V_n \cong \bigoplus_k V_k$ with $k = |m-n|, |m-n|+2, \dots, m+n$, and each summand is generated by a *highest-weight vector*, a K -eigenvector annihilated by E .

2.2 R-matrices and Yang–Baxter-type identities

Let V have dimension d and let R be an operator on $V \otimes V$, i.e. a $d^2 \times d^2$ matrix. On $V^{\otimes 3}$ one writes $R_{12} = R \otimes 1$, $R_{23} = 1 \otimes R$ and $R_{13} = P_{23} R_{12} P_{23}$, where P_{23} exchanges the second and

third factors. The quantum Yang–Baxter equation and, for the braided form $\check{R} = PR$, the braid relation read

$$R_{12}R_{13}R_{23} = R_{23}R_{13}R_{12}, \quad \check{R}_{12}\check{R}_{23}\check{R}_{12} = \check{R}_{23}\check{R}_{12}\check{R}_{23}; \quad (6)$$

the two are equivalent [4]. For the fundamental representation, $\check{R}$ moreover satisfies the Hecke relation

$$\check{R} - \check{R}^{-1} = (q - q^{-1}) \mathbb{1}, \quad \text{equivalently} \quad (\check{R} - q)(\check{R} + q^{-1}) = 0. \quad (7)$$

Neither relation refers to the Hopf structure. What ties an R-matrix on $V \otimes V$ to a coproduct is the intertwining condition

$$R \Delta(X) = \Delta^{\text{op}}(X) R \quad \text{for all generators } X, \quad (8)$$

which implies that $\check{R} = PR$ commutes with the action of $\Delta(X)$ and is therefore a module endomorphism of $V \otimes V$. A matrix can satisfy Eqs. (6) and (7) and yet fulfil Eq. (8) only for Δ^{op} in place of Δ ; Section 5.2 shows such a case.

2.3 $\mathbb{Z}_2$ -graded tensor products and $GL_q(2|1)$

A super vector space $V = V_0 \oplus V_1$ comes with a basis e_i of homogeneous vectors of parity $p(i) \in \{0, 1\}$; the basis vector $e_a \otimes e_b$ of $V \otimes V$ has parity $p(a) + p(b) \bmod 2$. An operator is *even* if it maps even vectors to even ones and odd to odd ones, i.e. if its matrix element between basis vectors of different parity vanishes. In tensor products the Koszul sign rule applies: exchanging homogeneous elements of parities a and b produces the sign $(-1)^{ab}$. Accordingly the flip P is replaced by the super-permutation

$$P_s(e_i \otimes e_j) = (-1)^{p(i)p(j)} e_j \otimes e_i, \quad (9)$$

and an even operator on $V \otimes V$ is placed on factors (i, j) of $V^{\otimes n}$ by conjugation with graded rather than ordinary flips. With R_{13} formed in this way, the *graded* Yang–Baxter equation has the form of Eq. (6).

For $GL_q(2|1)$, $V = \mathbb{C}^{2|1}$ with basis e_0, e_1 (even) and e_2 (odd); Ref. [7] numbers these vectors from 1. In the basis $e_a \otimes e_b$, ordered by $3a + b$, the R-matrix printed on page 261 of Ref. [7], as implemented in `R_matrix_GLq21`, is

$$R = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & q^2 & 0 & 1 - q^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & q & 0 & 0 & 0 & 1 - q^2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & q & 0 & 1 - q^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & q & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & q & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & q^2 \end{pmatrix}. \quad (10)$$

Its three off-diagonal entries couple $e_0 \otimes e_1$ with $e_1 \otimes e_0$, $e_0 \otimes e_2$ with $e_2 \otimes e_0$, and $e_1 \otimes e_2$ with $e_2 \otimes e_1$, pairs of equal parity, so R is even. It satisfies the graded Yang–Baxter equation but not the ungraded one (Sec. 5.1).

3 Design and workings

`quantum-group` is a pure-Python package of fifteen modules (about 2,100 lines, including docstrings) with no compiled extensions. Figure 1 shows the verification workflow that the modules implement; the design decisions behind each step are described below.

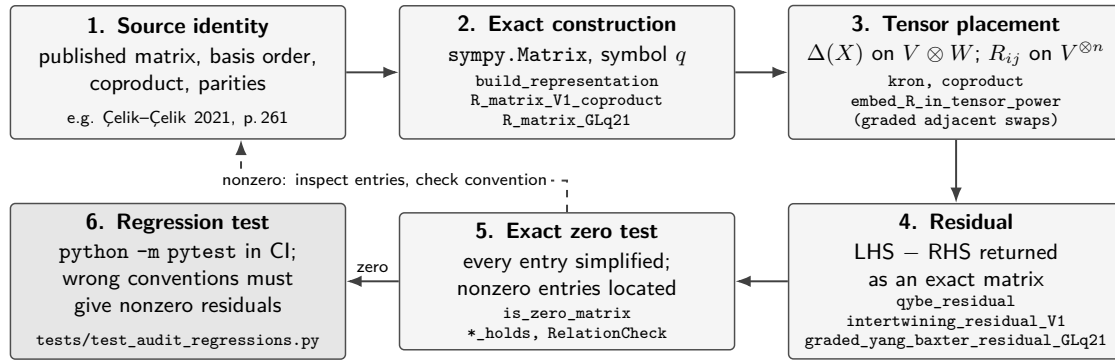


Figure 1: Verification workflow in quantum-group. A published identity and its conventions (1) are turned into exact SymPy matrices (2) and placed on tensor products through the coproduct or through graded adjacent swaps (3). The difference of the two sides is returned as an exact residual matrix (4) and tested entry by entry (5); a nonzero residual points to the entries, and hence the convention, that disagree. Zero residuals, and nonzero residuals for deliberately wrong conventions, are kept as regression tests (6). Function names are those of the package; `kron` is imported from `quantum_group.linalg`, all others from the package namespace.

117 3.1 Explicit matrices rather than an abstract algebra engine

118 The package contains no noncommutative rewriting system; the symbols in `generators.py`
 119 only display the relations. Every check runs on concrete representation matrices. This keeps
 120 each object directly comparable, entry by entry, with a printed matrix, and makes every in-
 121 termediate result an ordinary SymPy object that users can print, substitute into or pass to
 122 other SymPy code. The price is that a verification holds for the representations on which it is
 123 run, and that the cost grows with the dimension of the tensor product (Sec. 6.4). Identities
 124 that need a genuinely noncommutative graded algebra, such as the Gauss decomposition of
 125 $GL_q(2|1)$, are omitted rather than approximated with commuting symbols.

126 3.2 Residual-first verification

127 Each check follows one pattern: build the left-hand side, build the right-hand side, return their
 128 difference, and decide whether every entry is zero. Functions named `*_residual` (`qybe_`
 129 `residual`, `braid_relation_residual`, `intertwining_residual_V1`, `graded_yang_`
 130 `baxter_residual_GLq21`, and others) return the exact difference matrix; the correspond-
 131 ing `*_holds` functions apply `is_zero_matrix` to it; and `verify_on_representation`
 132 returns, for each defining relation, a `RelationCheck` record holding both the Boolean and
 133 the residual. Returning the residual rather than only a Boolean is what makes a failed check
 134 informative: the positions and values of the nonzero entries identify which basis vectors and
 135 which convention disagree (Sec. 5).

136 The zero test simplifies each entry with `sympy.simplify`. A True result means that every
 137 entry was reduced to the exact value 0; a False result is not by itself a proof of inequality,
 138 because `simplify` is not a decision procedure. For the rational functions of q produced by the
 139 package, a nonzero entry can be confirmed with `sympy.cancel`, which the regression tests
 140 do.

3.3 Conventions for $U_q(\mathfrak{sl}_2)$

For a highest weight n , the function `build_representation(n)` returns the $(n + 1)$ -dimensional module V_n as a `Representation` record with explicit matrices E , F , K and K^{-1} in the basis $v_0, \dots, v_n$, as in Eq. (5): K is diagonal, F is a lower shift and E carries the products of q -integers. `coproduct` and `tensor_product` realize the coproduct (2) with Kronecker products in the lexicographic basis order (left factor slow, right factor fast). `find_highest_weight_vectors` solves for the vectors annihilated by E in each weight space of $V_m \otimes V_n$, giving one highest-weight vector per Clebsch–Gordan summand.

Two R-matrix conventions. On $V_1 \otimes V_1$, in the basis $v_0 \otimes v_0, v_0 \otimes v_1, v_1 \otimes v_0, v_1 \otimes v_1$, the package provides

$$R = \begin{pmatrix} q & 0 & 0 & 0 \\ 0 & 1 & q - q^{-1} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & q \end{pmatrix}, \quad R_{21} = \begin{pmatrix} q & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & q - q^{-1} & 1 & 0 \\ 0 & 0 & 0 & q \end{pmatrix}, \quad (11)$$

where $R_{21} = PRP$ and P is the flip of the two factors. `R_matrix_V1` returns the historical upper-triangular matrix R , the image, rescaled by $q^{1/2}$, of Drinfeld’s universal R-matrix [1] for the coproduct opposite to Eq. (2); accordingly it satisfies $R \Delta^{\text{op}}(X) = \Delta(X)R$, i.e. it intertwines the *opposite* of the coproduct (2). Rather than change its output, which would silently alter results already computed with it, version 1.1.0 kept it unchanged and added `R_matrix_V1_coproduct` (R_{21}), `R_check_V1_coproduct` (PR_{21}) and `intertwining_residual_V1`, which satisfy $R_{21} \Delta(X) = \Delta^{\text{op}}(X)R_{21}$. Both conventions satisfy the QYBE, and both braided forms satisfy the braid relation and the Hecke relation $\check{R} - \check{R}^{-1} = (q - q^{-1})\mathbb{1}$, with eigenvalues q (multiplicity three) and $-q^{-1}$; they differ in which tensor-product submodules are the braid eigenspaces. Keeping both, with the convention stated in each docstring, lets earlier calculations be reproduced exactly while new work uses the compatible matrices. The same policy applies to names: a renamed function keeps its old name as a thin wrapper that emits a `DeprecationWarning`.

3.4 Graded tensor embeddings for $GL_q(2|1)$

For $GL_q(2|1)$ the basis parities $(0, 0, 1)$ are an explicit argument, and the graded flip P_s of Eq. (9) is built once, by `super_permutation_matrix`. `embed_R_in_tensor_power` places a $d^2 \times d^2$ operator on factors (i, j) of $V^{\otimes n}$ by first embedding it on the adjacent pair $(i, i+1)$ with Kronecker products and then conjugating by graded adjacent swaps $S_k = \mathbb{1}^{\otimes k} \otimes P_s \otimes \mathbb{1}^{\otimes (n-k-2)}$ until the second factor reaches position j . The Koszul sign rule is therefore encoded in a single matrix and reused for every placement, instead of being written out separately for R_{13} , R_{14} , R_{24} and so on. The construction is correct only for even (parity-preserving) operators, and the function checks this before embedding. Because the parity list is an argument, the same function also embeds operators on other super vector spaces, and an all-even parity list reproduces the ungraded placement. `R_matrix_GLq21` returns the 9×9 R-matrix (10) exactly as printed on page 261 of Ref. [7], and `graded_yang_baxter_residual_GLq21` forms $R_{12}R_{13}R_{23} - R_{23}R_{13}R_{12}$ on $V^{\otimes 3}$.

3.5 Exact parameters

The default parameter is a nonzero SymPy symbol, and the identities are rational in q , holding wherever their denominators are defined. Integers and exact rationals passed as q are converted to exact SymPy numbers, so `build_representation(2, 2)` yields entries such as

181 $5/2$ rather than 2.5 ; only floating-point values supplied by the user are approximate. q -integers
 182 and q -binomials are evaluated as Laurent polynomials, not as quotients, so that specializations
 183 with removable singularities, such as $q = -1$ or roots of unity, give the correct values instead
 184 of $0/0$.

185 3.6 Validation and failure modes

186 Public functions reject invalid input with specific errors rather than returning a misleading
 187 matrix: non-integral or negative highest weights, sizes and indices raise `TypeError` or
 188 `ValueError`; `verify_on_representation` rejects non-square or mismatched generator
 189 matrices and the singular values $q \in \{0, 1, -1\}$ of the defining commutator quotient; `qybe_`
 190 `residual` rejects matrices that are not $d^2 \times d^2$; and `embed_R_in_tensor_power` rejects
 191 invalid positions, parity lists and odd operators. Symbolic input is not restricted: any SymPy
 192 expression can be used as q .

193 3.7 Package structure

194 The modules form three layers. A small foundation provides the default symbol q (de-
 195 clared nonzero), q -integers, q -factorials and q -binomials (`utils`); the Kronecker product
 196 `kron/kron_list` and the zero-residual test `is_zero_matrix` (`linalg`); and internal
 197 argument checks (`_validation`). Construction modules build the matrices: $U_q(\mathfrak{sl}_2)$
 198 modules (`representations`), coproduct, counit and antipode (`hopf`), tensor products
 199 and Clebsch–Gordan vectors (`tensor`), $U_q(\mathfrak{sl}_2)$ R-matrices (`r_matrix`) and the $GL_q(2|1)$
 200 R-matrix, super-permutation and embeddings (`supergroup_gl21`). Verification functions
 201 sit next to the constructions they check and all use the same foundation. Auxiliary modules
 202 provide classical and root-of-unity specializations (`limits`), the crystal graph $B(n)$ as a Net-
 203 workX [14] graph (`crystal`), Matplotlib [15] weight and crystal plots (`visualization`)
 204 and a convenience facade, `QuantumGroupSL2`. The main functions are re-exported from the
 205 package namespace and listed in `__all__`.

206 4 Installation and usage

207 4.1 Requirements, installation and availability

208 `quantum-group` requires CPython 3.10 or later and runs on any operating system; it has no
 209 compiled code and needs no special hardware. It is tested in continuous integration on Linux,
 210 macOS and Windows (Sec. 6) and locally on Ubuntu 24.04 (x86-64). The runtime dependen-
 211 cies, installed automatically, are SymPy ≥ 1.10 [6], NetworkX ≥ 2.6 [14] and Matplotlib $\geq$
 212 3.5 [15]; continuous integration exercises the oldest supported versions (SymPy 1.10.1, Net-
 213 workX 2.6.3, Matplotlib 3.5.3). `pytest` [16] is needed only to run the test suite (extra `[test]`).

The version described here is 1.1.1, tagged as the GitHub Release v1.1.1. Its package
 code and test suite are those of version 1.1.0; release 1.1.1 adds the listings of this paper with
 their expected outputs, the data of the cross-check in Sec. 6.3, and continuous-integration jobs
 that run them. It is installed from the Python Package Index, or directly from the tag:

```
python -m pip install quantum-group==1.1.1
python -m pip install \
    "git+https://github.com/TerekliTahaBerk/quantum-groups@v1.1.1"
214 python -c "import quantum_group; print(quantum_group.__version__)" # 1.1.1
```

To run the tests and benchmarks, clone the repository and install it with the test extra:

```
git clone https://github.com/TerekliTahaBerk/quantum-groups.git
```

Table 1: Main public functions of quantum-group 1.1.1, grouped by task. All are importable from the package namespace.

Task	Functions
$U_q(\mathfrak{sl}_2)$ modules and relations	build_representation, verify_on_representation, all_relations_hold, symbolic_relations
Hopf structure	coproduct, counit, antipode, verify_coassociativity, verify_antipode, verify_all_hopf_axioms
Tensor products	tensor_product, find_highest_weight_vectors, cg_summands, cg_decomposition_summary, swap_matrix, kron_list
$U_q(\mathfrak{sl}_2)$ R-matrices	R_matrix_V1, R_matrix_V1_coproduct, R_check_V1, R_check_V1_coproduct, R_check_eigenvalues
Generic residuals	qybe_residual, qybe_holds, braid_relation_residual, braid_relation_holds, hecke_skein_relation_check, intertwining_residual_V1, intertwining_holds_V1, is_zero_matrix
$GL_q(2 1)$ and grading	R_matrix_GLq21, super_parity_gl21, super_permutation_matrix, embed_R_in_tensor_power, all_Rij_GLq21, graded_yang_baxter_residual_GLq21, graded_yang_baxter_holds_GLq21, local_ybe_on_four_tensor_GLq21, braid_far_commutativity_residual_GLq21
Specializations	q_integer, q_factorial, q_binomial, classical_limit, root_of_unity_substitution
Crystals and plots	build_crystal, e_tilde, f_tilde, plot_weight_diagram, plot_crystal_graph

```

cd quantum-groups && python -m pip install -e ".[test]"
python -m pytest                                     # 202 tests
215 python benchmarks/benchmark.py --repeats 3 --max-n 5    # Table 3

```

216 **Code availability.** Repository: <https://github.com/TerekliTahaBerk/quantum-groups> (MIT
217 licence). Release: [https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v](https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.1)
218 [1.1.1](https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.1). Package: <https://pypi.org/project/quantum-group/1.1.1/>. Archive: Zenodo [5],
219 a source archive of the release made from the GitHub Release; version 1.1.0 is archived as
220 doi:[10.5281/zenodo.23015576](https://doi.org/10.5281/zenodo.23015576) [17]. Listing 1 is examples/sample_verification.py,
221 and Listings 2–6 are in scipost/examples/ and scipost/crosscheck/; all six are part
222 of the release, each with its expected output.

223 4.2 Overview of the public interface

224 Table 1 lists the main functions by task; Appendix A gives their arguments and return values.
225 Functions that construct q -dependent objects accept an optional parameter `q_sym` (default:
226 the package symbol `q`); all return ordinary SymPy objects or plain Python records of them,
227 and each states its convention in the docstring.

228 4.3 Documentation and support

229 Documentation consists of the README (installation, quickstart, conventions, limitations),
230 module and function docstrings that state the conventions used (basis order, coprod-
231 uct, parities) and the relation each check tests, CONTRIBUTING.md, CHANGELOG.md,
232 an exploratory Jupyter notebook, and MANUSCRIPT_CODE_MAPPING.md, which links

the computational claims of the thesis to their implementation and tests. Questions, bug reports and feature requests are handled through the GitHub issue tracker (<https://github.com/TerekliTahaBerk/quantum-groups/issues>). CONTRIBUTING.md describes the development set-up, the validation command and the conventions for new checks (a `*_residual` function with a `*_holds` companion, input validation with a test for each error path, and a stated reference for any convention). The package is maintained by its author on a best-effort basis, with no institutional support or guaranteed response time; changes are recorded in CHANGELOG.md and versions follow semantic versioning.

5 Worked examples

The outputs below were produced with quantum-group 1.1.1, SymPy 1.14 and Python 3.11; each is stored next to its script and is compared automatically with a fresh run.

5.1 A first verification session

Listing 1 (examples/sample_verification.py) checks an installation in a few seconds. It verifies the defining relations on V_2 , the QYBE and coproduct intertwining of R_{21} , and the graded Yang–Baxter equation of $GL_q(2|1)$; it then repeats the last check with the $GL_q(2|1)$ R-matrix placed by ordinary, all-even swaps, as a negative control.

```
from quantum_group import (
    build_representation, verify_on_representation, R_matrix_V1_coproduct,
    qybe_holds, intertwining_holds_V1, R_matrix_GLq21,
    embed_R_in_tensor_power, graded_yang_baxter_residual_GLq21,
    is_zero_matrix)

rep = build_representation(2)          # V_2 of U_q(sl_2): 3x3 matrices
print("E on V_2:", rep.E.tolist())
checks = verify_on_representation(rep.E, rep.F, rep.K, rep.K_inv)
print("relations:", {k: c.holds for k, c in checks.items()})

R = R_matrix_V1_coproduct()           # R_21, matches the package coproduct
print("QYBE:", qybe_holds(R))
print("intertwines Delta:",
      [intertwining_holds_V1(X) for X in ("E", "F", "K", "K_inv")])

Y = graded_yang_baxter_residual_GLq21() # GL_q(2|1), parities (0, 0, 1)
print("graded YBE residual:", Y.shape, "zero:", is_zero_matrix(Y))

# Negative control: the same R placed with ordinary (all-even) swaps.
R12, R13, R23 = (embed_R_in_tensor_power(R_matrix_GLq21(), ij, 3, [0, 0, 0])
                  for ij in [(0, 1), (0, 2), (1, 2)])
bad = (R12 * R13 * R23 - R23 * R13 * R12).expand()
print("ungraded residual, nonzero entries:",
      {(i, j): x.factor()
       for (i, j), x in sorted(bad.todok().items()) if x != 0})
```

Listing 1: First verification session: examples/sample_verification.py.

Output (examples/sample_verification_expected.txt; the last line is wrapped here):

```
E on V_2: [[0, q + 1/q, 0], [0, 0, q + 1/q], [0, 0, 0]]
relations: {'R1': True, 'R2': True, 'R3': True, 'R4': True}
QYBE: True
intertwines Delta: [True, True, True, True]
graded YBE residual: (27, 27) zero: True
ungraded residual, nonzero entries: {(8, 24): 2*q**2*(q - 1)**2*(q + 1)**2,
```

250 (17, 25): 2*q**2*(q - 1)**2*(q + 1)**2}

251 The negative control shows why the grading matters: with ordinary swaps, exactly two entries
 252 of the 27×27 residual are nonzero, both equal to $2q^2(q-1)^2(q+1)^2$. Rows and columns are
 253 indexed lexicographically, $e_a \otimes e_b \otimes e_c \mapsto 9a + 3b + c$, so entry (8,24) couples $e_0 \otimes e_2 \otimes e_2$
 254 with $e_2 \otimes e_2 \otimes e_0$; both involve the odd vector e_2 twice, which is exactly where the Koszul sign
 255 $(-1)^{p(i)p(j)}$ differs from +1.

256 5.2 Application: which coproduct does an R-matrix intertwine?

257 This is the check that exposed the convention error in the author's thesis [8]. The thesis
 258 used the coproduct (2) together with the upper-triangular matrix R of Eq. (11). R satis-
 259 fies the QYBE and its braided form satisfies the braid and Hecke relations, so every Yang-
 260 Baxter-type test passed. The relation that fails is the one that ties an R-matrix to a coproduct,
 261 $R\Delta(X) = \Delta^{\text{op}}(X)R$. Listing 2 forms this residual for both matrices of Eq. (11) with public
 262 functions only, so that the same code can be pointed at any other 4×4 R-matrix.

```

import sympy as sp
from quantum_group import (build_representation, coproduct, swap_matrix,
                           R_matrix_V1, R_matrix_V1_coproduct, qybe_holds,
                           is_zero_matrix)

D = coproduct(build_representation(1)) # Delta(X) on V_1 (x) V_1, 4x4
P = swap_matrix(2)                    # ordinary flip of the two factors

def intertwining_residual(R, X):
    """R Delta(X) - Delta^op(X) R, with Delta^op(X) = P Delta(X) P."""
    return R * D[X] - P * D[X] * P * R

for name, R in [("R_matrix_V1", R_matrix_V1()),
                ("R_matrix_V1_coproduct", R_matrix_V1_coproduct())]:
    print(name, "- QYBE:", qybe_holds(R))
    for X in ("E", "F", "K"):
        res = intertwining_residual(R, X)
        nonzero = {ij: sp.factor(x) for ij, x in sorted(res.todok().items())
                    if sp.simplify(x) != 0}
        print(f" {X}:", "zero" if is_zero_matrix(res) else nonzero)

```

Listing 2: Testing an R-matrix against the coproduct:
 scipost/examples/coproduct_check.py.

Output (scipost/examples/coproduct_check_expected.txt; long lines wrapped here):

```

R_matrix_V1 - QYBE: True
  E: {(0, 1): (q - 1)*(q + 1), (0, 2): -(q - 1)*(q + 1)/q,
      (1, 3): (q - 1)*(q + 1)/q**2, (2, 3): -(q - 1)*(q + 1)/q}
  F: {(1, 0): (q - 1)*(q + 1)/q**2, (2, 0): -(q - 1)*(q + 1)/q,
      (3, 1): (q - 1)*(q + 1), (3, 2): -(q - 1)*(q + 1)/q}
  K: zero
R_matrix_V1_coproduct - QYBE: True
  E: zero
  F: zero
  K: zero

```

265 Both matrices pass the QYBE, but only R_{21} intertwines Δ . The residual of R is informative in
 266 three ways. It vanishes for K , as it must for any weight-preserving matrix, so the problem is
 267 not the diagonal part. Every nonzero entry carries the factor $(q-1)(q+1)$, so all of them
 268 vanish at $q = \pm 1$, where $q - q^{-1} = 0$ and R and R_{21} coincide; the discrepancy is a genuine
 269 q -deformation effect. And every nonzero entry couples one of the middle basis vectors $v_0 \otimes v_1$,

270 $v_1 \otimes v_0$ (indices 1 and 2), the block in which R and R_{21} differ, to $v_0 \otimes v_0$ or $v_1 \otimes v_1$. The same
 271 residual with Δ and Δ^{op} exchanged vanishes for R , confirming that R belongs to the opposite
 272 coproduct. The package now exposes this check as `intertwining_residual_V1`, and the
 273 test suite keeps the failing case as a negative control.

274 5.3 Locating a transcription error

275 A matrix typed from a paper can be passed to `qybe_residual` directly. Listing 3 introduces
 276 a single sign error in R of Eq. (11), writing $q + q^{-1}$ for $q - q^{-1}$.

```
import sympy as sp
from quantum_group import q, qybe_residual, is_zero_matrix

# R-matrix typed from a source with one mistyped sign: q + 1/q at (1, 2),
# where the source (the package's R_matrix_V1) has q - 1/q.
R_typed = sp.Matrix([[q, 0, 0, 0],
                    [0, 1, q + 1/q, 0],
                    [0, 0, 1, 0],
                    [0, 0, 0, q]])
Y = qybe_residual(R_typed) # exact 8x8 residual
print("QYBE holds:", is_zero_matrix(Y))
print("nonzero entries:", {ij: sp.factor(x) for ij, x in sorted(Y.todok().items())
                          if sp.simplify(x) != 0})
```

Listing 3: A mistyped entry: `scipost/examples/transcription_check.py`.

Output (`scipost/examples/transcription_check_expected.txt`):

```
QYBE holds: False
nonzero entries: {(1, 4): -2*(q**2 + 1)/q, (3, 6): 2*(q**2 + 1)/q}
```

279 Only two of the 64 entries are nonzero. In the basis $e_a \otimes e_b \otimes e_c \mapsto 4a + 2b + c$ of $V_1^{\otimes 3}$, they
 280 couple $v_0 \otimes v_0 \otimes v_1$ with $v_1 \otimes v_0 \otimes v_0$ and $v_0 \otimes v_1 \otimes v_1$ with $v_1 \otimes v_1 \otimes v_0$, so the search for the error
 281 can start from the entries that move a v_1 between the first and third factors. A floating-point
 282 check at a sample value of q would report a failure as well, but not the exact form of the error.

283 5.4 Clebsch–Gordan decomposition

284 Listing 4 decomposes $V_2 \otimes V_1 \cong V_3 \oplus V_1$ by computing one highest-weight vector per summand
 285 in the tensor-product basis $v_i \otimes w_j \mapsto 2i + j$.

```
from quantum_group import (build_representation, tensor_product,
                          find_highest_weight_vectors, cg_summands)

T = tensor_product(build_representation(2), build_representation(1))
print("summands V_k, k =", cg_summands(2, 1))
for k, v in find_highest_weight_vectors(T):
    print(f"k = {k}:", list(v))
```

Listing 4: Highest-weight vectors of $V_2 \otimes V_1$: `scipost/examples/clebsch_gordan.py`.

Output (`scipost/examples/clebsch_gordan_expected.txt`):

```
summands V_k, k = [1, 3]
k = 3: [1, 0, 0, 0, 0, 0]
286 k = 1: [0, (-q**2 - 1)/q**3, 1, 0, 0, 0]
```

287 The highest-weight vector of the V_1 summand is $v_1 \otimes w_0 - q^{-3}(q^2 + 1)v_0 \otimes w_1$; its q -dependent
 288 coefficient is fixed by the coproduct (2): with Δ^{op} it would be $-(q^2 + 1)$ instead of $-q^{-3}(q^2 + 1)$,
 289 which is why the coproduct formulas are stated in the documentation of the `hopf` and `tensor`
 290 modules and in the README.

291 5.5 Graded identities on four factors and a reordered basis

292 Listing 5 runs two further graded checks for $GL_q(2|1)$. The first two lines verify the graded
 293 Yang–Baxter equation on every triple of factors of $V^{\otimes 4}$, with all placements R_{ij} built by graded
 294 adjacent swaps, and the commutation of the operators R_{12} and R_{34} on disjoint factors, an
 295 81×81 identity. The second part tests the parity list as an argument. It rewrites the R-
 296 matrix (10) in the reordered basis $f = (e_2, e_0, e_1)$, in which the odd vector comes first, and
 297 repeats the graded Yang–Baxter check once with the parities $(1, 0, 0)$ of the new basis and once
 298 with the stale list $(0, 0, 1)$ of the old one.

```
import sympy as sp
from quantum_group import (R_matrix_GLq21, embed_R_in_tensor_power,
                           local_ybe_on_four_tensor_GLq21,
                           braid_far_commutativity_residual_GLq21, is_zero_matrix)
from quantum_group.linalg import kron

print("local YBE on  $V^{\otimes 4}$ :", local_ybe_on_four_tensor_GLq21())
print("R12 R34 = R34 R12:",
      is_zero_matrix(braid_far_commutativity_residual_GLq21()))

# Reorder the basis so that the odd vector comes first: f = (e_2, e_0, e_1).
perm = [2, 0, 1]
Pi = sp.Matrix(3, 3, lambda k, i: int(perm[k] == i)) # new = Pi * old
R_new = kron(Pi, Pi) * R_matrix_GLq21() * kron(Pi, Pi).T
for parity in ([1, 0, 0], [0, 0, 1]): # updated / stale
    R12, R13, R23 = (embed_R_in_tensor_power(R_new, ij, 3, parity)
                     for ij in [(0, 1), (0, 2), (1, 2)])
    Y = R12 * R13 * R23 - R23 * R13 * R12
    print("parities", parity, "graded YBE holds:", is_zero_matrix(Y),
          "| nonzero entries:", sum(sp.simplify(x) != 0 for x in Y))
```

Listing 5: Graded identities on $V^{\otimes 4}$ and in a reordered basis:
 scipost/examples/graded_four_factors.py.

Output (scipost/examples/graded_four_factors_expected.txt; the first line is wrapped here):

```
local YBE on  $V^{\otimes 4}$ : {(0, 1, 2): True, (0, 1, 3): True, (0, 2, 3): True, (1, 2,
3): True}
R12 R34 = R34 R12: True
parities [1, 0, 0] graded YBE holds: True | nonzero entries: 0
300 parities [0, 0, 1] graded YBE holds: False | nonzero entries: 4
```

301 With the parity list that matches the basis, the graded Yang–Baxter equation holds in the re-
 302 ordered basis as well; the Koszul signs are recomputed from the list, not from a fixed position
 303 of the odd vector. Reordering the basis but keeping the old parities, a mistake of the same
 304 kind as the transcription errors of Sec. 1, leaves four nonzero entries in the 27×27 residual.
 305 The same call therefore serves any finite-dimensional super vector space whose parities are
 306 known.

307 6 Testing and benchmarks

308 6.1 Test suite and continuous integration

309 Every change is checked by one command, `python -m pytest`, which runs the test suite
 310 with `pytest` [16] together with the package doctests. At version 1.1.1, as at 1.1.0, the suite
 311 comprises 198 test cases in ten files and four doctests (202 in total, all passing). GitHub
 312 Actions runs it on every push and pull request on Python 3.10, 3.11, 3.12 and 3.13; once with

Table 2: Evidence provided by the test suite. Entries in the last column abbreviate `tests/test_name.py`. “Negative control” marks tests that apply a known wrong convention and require a nonzero residual.

Evidence	What is checked	Test file
Exact entries	Every nonzero entry of the $9 \times 9 GL_q(2 1)$ R-matrix against the display on printed page 261 of Ref. [7]; exact entries of the $4 \times 4 U_q(\mathfrak{sl}_2)$ R-matrix; shift and weight structure of V_n	<code>audit_regressions,</code> <code>r_matrix,</code> <code>representations</code>
Coproduct convention	$R_{21}\Delta(X) = \Delta^{\text{op}}(X)R_{21}$ for $X = E, F, K, K^{-1}$; negative control: the historical matrix fails for E and F	<code>audit_regressions</code>
Clebsch–Gordan bases	Complete change of basis for $V_1 \otimes V_1, V_2 \otimes V_2, V_3 \otimes V_2$: invertible over $\mathbb{Q}(q)$ and intertwining all four generators; braid eigenvalues q and $-q^{-1}$ on the V_2 and V_0 summands	<code>audit_regressions,</code> <code>tensor</code>
Graded embeddings	All six placements R_{ij} on $V^{\otimes 4}$ equal an independently written signed basis-action formula that builds no swap matrix; disjoint placements commute	<code>audit_regressions,</code> <code>supergroup_gl21</code>
Grading	Graded YBE residual (27×27) is zero; negative control: with the ordinary flip exactly two entries are nonzero, with their values fixed	<code>supergroup_gl21,</code> <code>audit_regressions</code>
Exact parameters	No floating-point atoms for integer, rational, $q = -1$ and $q = i$ inputs; Laurent-polynomial q -integers and q -binomials agree with generic formulas after substitution; root-of-unity orders validated	<code>audit_regressions,</code> <code>limits</code>
Algebraic identities	Defining relations on V_n ; coassociativity, counit and antipode on generators; QYBE, braid and Hecke relations	<code>relations, hopf,</code> <code>r_matrix</code>
Input validation	Every validation path raises the documented error; shared helpers agree with <code>sympy.kronecker_product</code>	<code>validation_and_</code> <code>linalg</code>

the oldest supported dependency versions (SymPy 1.10.1, NetworkX 2.6.3, Matplotlib 3.5.3 on Python 3.10); and, after building and installing a wheel, runs the README quickstart and Listings 1–6 from outside the source tree, comparing each listing’s output with its stored copy. A further job repeats the wheel installation, the quickstart, Listing 1 and the full test suite on Linux, macOS and Windows runners (Python 3.12), and the job with the oldest dependencies also runs Listings 1–6; their outputs are the same with SymPy 1.14 on Python 3.12 and with SymPy 1.10.1 on Python 3.10. Table 2 summarizes what the tests establish. Besides algebraic identities, the suite benchmarks the implementation against published reference values: every nonzero entry of the $GL_q(2|1)$ R-matrix is compared with the printed source [7], and the normalization of the $U_q(\mathfrak{sl}_2)$ R-matrix is recomputed from Drinfeld’s universal R-matrix [1]. Section 6.3 adds a comparison with independently computed data from GAP’s QuaGroup.

6.2 Negative controls and independent re-derivations

A suite that only checks that correct identities hold cannot show that the checks are able to fail. Several tests therefore apply a known wrong convention (the opposite coproduct, the ordinary instead of the graded flip, a mutated counit) and require the residual to be nonzero, in some cases at specified entries with specified values: the two nonzero entries of the ungraded residual in Listing 1 are fixed by a test, and the failure of the historical R-matrix to intertwine the coproduct (Listing 2) is kept as a test requiring a nonzero residual. These tests fix the scientific content of the conventions, not only the behaviour of the code. Two further tests are

independent re-derivations: the four-factor placements are compared with a formula that acts on basis vectors directly, and the normalization of the fundamental R-matrix is recomputed from Drinfeld's universal R-matrix [1].

6.3 Cross-check against GAP's QuaGroup

The tests above compare the package with published formulas. To compare it with existing software as well, the module matrices of V_1, V_2, V_3 , the action of $U_q(\mathfrak{sl}_2)$ on $V_n \otimes V_n$ ($n = 1, 2$) and the R-matrices $\text{RMatrix}(V_n)$ ($n = 1, 2$) were computed with QuaGroup 1.8.4 [10, 11] in GAP 4.12.1 [9], the package behind SageMath's QuantumGroup interface [12], and exported by the GAP script `scipost/crosscheck/quagroup_export.g`. The exported file (`quagroup_data.txt`, in the repository) is read by Listing 6, which tests QuaGroup's data with the residual functions of `quantum-group`. GAP is needed only to regenerate the data.

```

import re
import sympy as sp
from quantum_group import (q, build_representation, coproduct, swap_matrix,
                           verify_on_representation, R_matrix_V1_coproduct,
                           qybe_holds, braid_relation_residual, is_zero_matrix)
from quantum_group.linalg import kron

G = {} # matrices exported from QuaGroup by quagroup_export.g
for line in open(__file__.replace("quagroup_crosscheck.py", "quagroup_data.txt")):
    if m := re.match(r"(\w+) = ([\.*\d])$", line.strip()):
        G[m[1]] = sp.Matrix(sp.sympify(m[2].replace("^", "**"), {"q": q}))
Z = is_zero_matrix

for n in (1, 2, 3):
    # QuaGroup's V_n vs the package's V_n
    E, F, K, Ki = (G[f"{X}_V{n}"] for X in ("E", "F", "K", "K_inv"))
    ok = all(c.holds for c in verify_on_representation(E, F, K, Ki).values())
    D = sp.diag(*[1 / sp.prod([sum(q**(k - 1 - 2*i) for i in range(k))
                              for k in range(1, j + 1)]) for j in range(n + 1)])
    r = build_representation(n) # D = diag(1/[j!]): divided powers
    print(f"V_{n}: relations {ok}, equal after D:",
          Z(D * E * D.inv() - r.E) and Z(D * F * D.inv() - r.F) and K == r.K)

for n in (1, 2):
    # QuaGroup's V_n (x) V_n and R-matrix
    E, F, K, Ki = (G[f"{X}_V{n}"] for X in ("E", "F", "K", "K_inv"))
    I, P = sp.eye(n + 1), swap_matrix(n + 1)
    Delta = {"E": kron(E, I) + kron(K, E), "F": kron(F, Ki) + kron(I, F),
            "K": kron(K, K)}
    print(f"V_{n} (x) V_{n}: action = package coproduct:",
          all(Z(Delta[X] - G[f"{X}_T{n}"]) for X in "EFK"))
    for label, th in (("as printed", G[f"RMatrix_V{n}"]),
                     ("transposed", G[f"RMatrix_V{n}"].T)):
        commutes = all(Z(th * P * G[f"{X}_T{n}"] - G[f"{X}_T{n}"] * th * P)
                       for X in ("E", "F", "K"))
        print(f" RMatrix {label}: QYBE {qybe_holds(th)},",
              f"braid {Z(braid_relation_residual(th * P))}",
              f"theta P commutes with Delta {commutes}")

th, R21 = G["RMatrix_V1"].T, R_matrix_V1_coproduct()
print("theta = q * R_21(1/q):", Z(th - q * R21.subs(q, 1 / q)))
D2 = sp.diag(1, 1, 1 / (q + 1 / q))
th2 = kron(D2, D2) * G["RMatrix_V2"].T * kron(D2, D2).inv() # package basis
Rc2, cp = th2 * swap_matrix(3), coproduct(build_representation(2))
print("V_2 R-matrix in package basis: QYBE", qybe_holds(th2), "| commutes",
      all(Z(Rc2 * cp[X] - cp[X] * Rc2) for X in ("E", "F", "K")),
      "| eigenvalues", {sp.factor(k): v for k, v in Rc2.eigenvals().items()})

```

Listing 6: Testing QuaGroup’s data with quantum-group:
 scipost/crosscheck/quagroup_crosscheck.py.

Output (scipost/crosscheck/quagroup_crosscheck_expected.txt; the last line is wrapped here):

```
V_1: relations True, equal after D: True
V_2: relations True, equal after D: True
V_3: relations True, equal after D: True
V_1 (x) V_1: action = package coproduct: True
  RMatrix as printed: QYBE True, braid True, theta P commutes with Delta False
  RMatrix transposed: QYBE True, braid True, theta P commutes with Delta True
V_2 (x) V_2: action = package coproduct: True
  RMatrix as printed: QYBE True, braid True, theta P commutes with Delta False
  RMatrix transposed: QYBE True, braid True, theta P commutes with Delta True
theta = q * R_21(1/q): True
V_2 R-matrix in package basis: QYBE True | commutes True
| eigenvalues {q**(-2): 5, -q**2: 3, q**4: 1}
```

The two systems agree on the mathematics and differ in three conventions, each of which the residuals detect. (i) QuaGroup’s modules satisfy the relations (1) and have the same K , but its basis is the divided-power basis $F^{(k)}v_0 = F^k v_0 / [k]!$; after conjugation with $D = \text{diag}(1/[k]!)$ its E and F equal the package’s. The action on $V_n \otimes V_n$ is exactly the coproduct (2). (ii) GAP writes matrices in the row-vector convention; QuaGroup’s source builds RMatrix row by row from the images of the basis vectors. Read as printed, RMatrix(V_n) still satisfies the QYBE, which is invariant under transposition, and hence the braid relation, but θP does not commute with the coproduct; only the intertwining test exposes the misreading. Transposed, the matrix passes all three tests. (iii) For V_1 the transposed matrix is $\theta = q R_{21}(q^{-1})$, so that $\theta P = q \check{R}^{-1}$ with $\check{R}$ the braiding of R_check_V1_coproduct: QuaGroup’s braiding is the inverse of the package’s, rescaled by q , and has eigenvalues 1 (multiplicity three) and $-q^2$ instead of q and $-q^{-1}$. Finally, QuaGroup’s 9×9 R-matrix of V_2 , which quantum-group does not provide, satisfies the QYBE and commutes with the package coproduct once transported to the package basis; its braid eigenvalues q^{-2} , $-q^2$ and q^4 have multiplicities 5, 3 and 1, the dimensions of the summands V_4 , V_2 and V_0 of $V_2 \otimes V_2$. External R-matrices can thus be brought into the package’s conventions and verified there, which is the extension route described in Sec. 7.

6.4 Performance benchmarks

Operators on $V^{\otimes n}$ are stored as dense symbolic matrices, $3^n \times 3^n$ for $GL_q(2|1)$, which keeps every entry inspectable but grows exponentially. Table 3 reproduces the recorded benchmark (benchmarks/results.md), generated by the command given in Sec. 4.1. Tensor powers $n \geq 6$ have not been benchmarked. A sparse or tensor-network backend could reach larger n , at the cost of the entry-by-entry inspectability that motivates the design; the calculations the package targets involve at most four or five factors.

7 Scope, limitations and extensions

Scope and limitations. A zero residual establishes an identity on the particular finite-dimensional representation used; it is not a proof in the abstract Hopf algebra, and the package is not a theorem prover. It does not implement general quantum groups. For $GL_q(2|1)$ it implements the R-matrix, the graded Yang–Baxter equation and tensor placements, not the RTT relations, the Hopf superalgebra structure or the Gauss decomposition of Ref. [7]. It verifies the Hecke relation but does not compute a Markov trace or the Jones

Table 3: Wall-clock times (mean and minimum of three runs after a first, separately reported run) from benchmarks/results.md; the smallest case is dominated by timing noise (standard deviation 0.080 s). Environment: Python 3.11, SymPy 1.14, one core of a 2.1 GHz Intel Xeon processor, 27 September 2026.

Computation	n	Matrix size	Mean (s)	Min (s)
QYBE of R for $U_q(\mathfrak{sl}_2)$ (qybe_holds)	3	8×8	0.075	0.028
Graded YBE residual for $GL_q(2 1)$	3	27×27	0.025	0.024
Graded YBE, residual and zero test	3	27×27	0.060	0.055
All placements R_{ij} (all_Rij_GLq21)	4	81×81	0.977	0.954
Local YBE on every triple (local_ybe_on_four_tensor_GLq21)	4	81×81	1.556	1.540
All placements R_{ij} (all_Rij_GLq21)	5	243×243	14.616	14.276
Graded YBE on one triple, incl. embedding	5	243×243	15.221	14.799

polynomial. It works with constant R-matrices; spectral-parameter-dependent solutions of the Yang–Baxter equation are not built in. Its root-of-unity helpers substitute special values into the generic construction; they do not implement the small quantum group. The crystal graph $B(n)$ is a combinatorial model rather than a crystal-basis algorithm.

Reuse. The extension points are ordinary Python functions that accept and return SymPy matrices, and the package is small enough to read in full. Any $d^2 \times d^2$ matrix can be passed to `qybe_residual` or `braid_relation_residual` (Sec. 5.3); the intertwining test of Listing 2 works for any R-matrix, module or coproduct formula; and `verify_on_representation` accepts any four square matrices, so alternative bases or normalizations of V_n can be checked directly. `super_permutation_matrix` and `embed_R_in_tensor_power` take an arbitrary parity list and can place any even operator on factors (i, j) of $V^{\otimes n}$ for other $\mathbb{Z}_2$ -graded spaces, such as $\mathbb{C}^{m|n}$ with a different ordering of even and odd basis vectors. Because each check returns an exact matrix, an identity from a paper or thesis becomes a pytest test in a few lines, with a known wrong convention kept alongside it as a negative control; `tests/test_audit_regressions.py` contains worked examples of this pattern. Outputs can be exported with `sympy.latex` or compared against matrices computed elsewhere, for example in SageMath, by forming a residual.

Where reuse requires new work. Moving from $\mathfrak{sl}_2$ to general semisimple Lie algebras would need root data, PBW bases and higher-rank modules, which QuaGroup and SageMath already provide. Implementing the RTT relations or the Hopf superalgebra of $GL_q(2|1)$ would need a noncommutative graded algebra rather than matrices. The dense matrix representation limits practical calculations to about five tensor factors of a three-dimensional space. These directions are possible but would be substantial new software rather than reuse of the existing layer. A lighter route is to import explicit matrices computed elsewhere, as done for the V_2 R-matrix of QuaGroup in Sec. 6.3, and verify them with the existing residual and embedding functions.

8 Conclusion

quantum-group turns convention-sensitive identities of quantum-group theory into exact residual matrices that can be inspected entry by entry and kept as regression tests. Its value lies less in the range of algebras it covers, which is deliberately small, than in making the

conventions explicit: basis order, coproduct and grading are fixed in documented functions, the Koszul sign rule is encoded once for every tensor placement, and every check can fail informatively. The thesis audit of Sec. 5.2 shows that this matters in practice: an R-matrix that passes every Yang–Baxter-type test can still belong to the wrong coproduct, and only a residual tied to the coproduct reveals it. The comparison with QuaGroup in Sec. 6.3 shows the same effect across software: the two systems agree on the mathematics, and the residuals locate exactly where their conventions for bases, matrix orientation and braiding differ. Natural next steps are further explicit rank-one and graded R-matrices, which reuse the existing residual and embedding layer unchanged, and a sparse backend for larger tensor powers.

Acknowledgements

The author thanks Prof. Dr. Salih Çelik for supervising the undergraduate thesis from which this software originated.

Author contributions T.B.T. is the sole author: T.B.T. designed and wrote the original implementation; decided on, reviewed and integrated all later changes to the software, including AI-assisted ones; and prepared and approved the manuscript.

Funding information This work received no specific funding.

Competing interests The author has no competing interests to declare.

Use of generative AI No generative AI tools were used during the original development and thesis work (April–June 2026). In September 2026, Claude Code (Anthropic) was used for packaging and continuous-integration configuration, the benchmark script, the shared matrix helpers, input validation and their tests, the sample-verification example, the worked-example and cross-check scripts of this paper, documentation, translating Turkish docstrings and comments into English, a mathematical and technical audit that led to the parameter-handling corrections, the coproduct-compatible R-matrix functions and the audit regression tests, and drafting of this manuscript. The model was Claude Opus 5.5, except for part of the manuscript and submission-material preparation, which used Claude Sonnet 5; commits produced with Claude Code are authored or co-authored by “Claude” in the Git history, with the model named in the commit trailer. No other AI tool was used. Every AI-assisted code change is exercised by the test suite, and every AI-assisted mathematical claim was checked by an executable symbolic test or against the primary source, in particular the published Çelik–Çelik article. The author reviewed and validated all AI-assisted output, made the scientific and design decisions, and takes responsibility for the software and this paper.

A Reference of the public interface

This appendix lists the main public functions of `quantum-group 1.1.1` with their arguments and return values; the docstrings give the full conventions. Unless stated otherwise, functions are imported from the package namespace (`from quantum_group import ...`), matrices are exact `sympy.Matrix` objects in the column convention, and tensor-product bases are ordered lexicographically with the left factor slowest. Functions whose result depends on q take a final optional argument `q_sym`, which defaults to the package symbol `q` and is omitted below. (q is declared nonzero; SymPy’s nonzero assumption also makes it real, which does

not affect identities between rational functions of q .) A `*_holds` function returns True exactly when the corresponding `*_residual` matrix passes `is_zero_matrix`.

Records.

- `Representation`: fields `n`, `dim`, `E`, `F`, `K`, `K_inv`, `weights`.
- `TensorRepresentation`: fields `m`, `n`, `dim`, `E`, `F`, `K`, `K_inv` (action on $V_m \otimes V_n$).
- `RelationCheck`: fields `name`, `holds`, `residual`; `HopfAxiomCheck`: fields `name`, `holds`, `detail`.

Modules and relations (Sec. 2.1).

- `build_representation(n)` → `Representation`: the module V_n of Eq. (5); `n` a nonnegative integer.
- `verify_on_representation(E, F, K, K_inv)` → dict: keys "R1"–"R4" for the relations (1), values `RelationCheck`. Accepts any four square matrices of equal size; rejects $q \in \{0, 1, -1\}$.
- `all_relations_hold(checks)` → bool; `symbolic_relations()` → list of (name, lhs, rhs) for display.

Hopf structure (Eqs. (2), (3)).

- `coproduct(rep)` → dict of the matrices $\Delta(X)$ on $V \otimes V$, keys "E", "F", "K", "K_inv".
- `counit()` → dict of $\varepsilon(X)$; `antipode(rep)` → dict of the matrices $S(X)$ on V .
- `verify_coassociativity(rep)` and `verify_antipode(rep)` → dict of `HopfAxiomCheck`.
- `verify_all_hopf_axioms(rep)` → the same checks, grouped by axiom.

Tensor products.

- `tensor_product(repA, repB)` → `TensorRepresentation`, the action through Δ .
- `find_highest_weight_vectors(T)` → list of (k, vector), one highest-weight vector per summand V_k .
- `cg_summands(m, n)` → list of the k in $V_m \otimes V_n \cong \bigoplus V_k$;
`cg_decomposition_summary(m, n)` → str.
- `swap_matrix(d)` → the flip P on $\mathbb{C}^d \otimes \mathbb{C}^d$; `kron_list(mats)` → left-to-right Kronecker product; `kron(A, B)` (from `quantum_group.linalg`).

$U_q(\mathfrak{sl}_2)$ R-matrices (Sec. 3.3).

- `R_matrix_V1()` → R of Eq. (11) (intertwines Δ^{op}); `R_check_V1()` → PR .
- `R_matrix_V1_coproduct()` → $R_{21} = PRP$ (intertwines Δ);
`R_check_V1_coproduct()` → PR_{21} .
- `R_check_eigenvalues()` → dict eigenvalue → multiplicity;
`hecke_skein_relation_check()` → dict with the Boolean of Eq. (7).
- `intertwining_residual_V1(X)` → $R_{21}\Delta(X) - \Delta^{\text{op}}(X)R_{21}$ for X in "E", "F", "K", "K_inv"; `intertwining_holds_V1(X)` → bool.

483 Generic residuals (any $d^2 \times d^2$ matrix).

- 484 • `qybe_residual(R)` $\rightarrow R_{12}R_{13}R_{23} - R_{23}R_{13}R_{12}$ on $V^{\otimes 3}$ (ungraded placement);
- 485 `qybe_holds(R)` $\rightarrow$ bool.
- 486 • `braid_relation_residual(Rc)` $\rightarrow \check{R}_{12}\check{R}_{23}\check{R}_{12} - \check{R}_{23}\check{R}_{12}\check{R}_{23}$ for a braided matrix Rc;
- 487 `braid_relation_holds(Rc)` $\rightarrow$ bool.
- 488 • `is_zero_matrix(M)` $\rightarrow$ True if every entry simplifies to exactly 0 (Sec. 3.2).

489 Grading and $GL_q(2|1)$ (Sec. 2.3).

- 490 • `super_parity_gl21()` $\rightarrow [0, 0, 1]$; `super_permutation_matrix(parity)`
- 491 $\rightarrow P_s$ of Eq. (9) for any parity list.
- 492 • `embed_R_in_tensor_power(R, (i, j), n, parity)` $\rightarrow$ the placement R_{ij} on
- 493 $V^{\otimes n}$ (zero-based, $i < j$) by graded adjacent swaps; rejects odd operators, invalid
- 494 positions and parity lists.
- 495 • `R_matrix_GLq21()` $\rightarrow$ Eq. (10); `all_Rij_GLq21(n)` $\rightarrow$ dict $(i, j) \rightarrow R_{ij}$ on $V^{\otimes n}$.
- 496 • `graded_yang_baxter_residual_GLq21()` $\rightarrow 27 \times 27$ residual;
- 497 `graded_yang_baxter_holds_GLq21()` $\rightarrow$ bool.
- 498 • `local_ybe_on_four_tensor_GLq21()` $\rightarrow$ dict triple $\rightarrow$ bool;
- 499 `braid_far_commutativity_residual_GLq21()` $\rightarrow R_{12}R_{34} - R_{34}R_{12}$ (81×81).

500 Specializations, crystals and plots.

- 501 • `q_integer(n)`, `q_factorial(n)`, `q_binomial(n, k)` $\rightarrow$ Laurent polynomials in
- 502 q ; `classical_limit(expr)` $\rightarrow$ the limit $q \rightarrow 1$.
- 503 • `root_of_unity_substitution(rep, N)` $\rightarrow$ dict with E^N, F^N, K^{2N} at $q = e^{2\pi i/N}$
- 504 ($N > 2$), substituted into the generic matrices.
- 505 • `build_crystal(n)` $\rightarrow$ NetworkX DiGraph of $B(n)$ with string node labels;
- 506 `crystal_nodes(n)` $\rightarrow$ list of CrystalNode records (index, weight, label).
- 507 • `e_tilde(node, n)`, `f_tilde(node, n)` $\rightarrow$ CrystalNode or None, for a
- 508 CrystalNode from `crystal_nodes(n)`.
- 509 • `plot_weight_diagram(n)`, `plot_crystal_graph(n)` $\rightarrow$ Matplotlib Figure.

510 References

- 511 [1] V. G. Drinfeld, *Quantum groups*, In *Proceedings of the International Congress of Mathe-*
- 512 *maticians, Berkeley, California, 1986*, vol. 1, pp. 798–820. American Mathematical Soci-
- 513 *ety*, Providence, RI, [https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM19](https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM1986.1/ICM1986.1.ocr.pdf)
- 514 [86.1/ICM1986.1.ocr.pdf](https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM1986.1/ICM1986.1.ocr.pdf) (1987).
- 515 [2] M. Jimbo, *A q -difference analogue of $U(\mathfrak{g})$ and the Yang–Baxter equation*, Lett. Math.
- 516 *Phys.* **10**, 63 (1985), doi:[10.1007/BF00704588](https://doi.org/10.1007/BF00704588).
- 517 [3] M. Jimbo, *A q -analogue of $U(\mathfrak{gl}(N+1))$, Hecke algebra, and the Yang–Baxter equation*,
- 518 *Lett. Math. Phys.* **11**, 247 (1986), doi:[10.1007/BF00400222](https://doi.org/10.1007/BF00400222).
- 519 [4] C. Kassel, *Quantum Groups*, vol. 155 of *Graduate Texts in Mathematics*, Springer, New
- 520 *York*, doi:[10.1007/978-1-4612-0783-2](https://doi.org/10.1007/978-1-4612-0783-2) (1995).
- 521 [5] T. B. Terekli, *quantum-group: Exact SymPy verification of explicit $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$*
- 522 *matrix identities*, GitHub Release (archived on Zenodo), Version 1.1.1, [https://github.c](https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.1)
- 523 [om/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.1](https://github.com/TerekliTahaBerk/quantum-groups/releases/tag/v1.1.1) (2026).

- [6] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig *et al.*, *SymPy: symbolic computing in Python*, PeerJ Comput. Sci. **3**, e103 (2017), doi:[10.7717/peerj-cs.103](https://doi.org/10.7717/peerj-cs.103).
- [7] S. Çelik and S. A. Çelik, *A new quantum supergroup and its Gauss decomposition*, Rep. Math. Phys. **88**, 259 (2021), doi:[10.1016/S0034-4877\(21\)00073-2](https://doi.org/10.1016/S0034-4877(21)00073-2).
- [8] T. B. Terekli, *Quantum grup yapılarının Python ortamında modellenmesi [Modelling quantum group structures in Python]*, Undergraduate thesis, Department of Mathematics, Yıldız Technical University, Istanbul, <https://github.com/TerekliTahaBerk/quantum-groups/tree/main/thesis> (2026).
- [9] The GAP Group, *GAP – Groups, Algorithms, and Programming*, Version 4.12.1, <https://www.gap-system.org> (2022).
- [10] W. A. de Graaf, *QuaGroup: computations with quantum groups (GAP package)*, Version 1.8.4, <https://github.com/gap-packages/quagroup> (2024).
- [11] W. A. de Graaf, *Computing with quantized enveloping algebras: PBW-type bases, highest-weight modules and R-matrices*, J. Symb. Comput. **32**, 475 (2001), doi:[10.1006/jSCO.2001.0479](https://doi.org/10.1006/jSCO.2001.0479).
- [12] The Sage Developers, *SageMath, the Sage Mathematics Software System*, doi:[10.5281/zenodo.8042260](https://doi.org/10.5281/zenodo.8042260), Version 10.9, <https://www.sagemath.org> (2026).
- [13] G. Benkart, S.-J. Kang and M. Kashiwara, *Crystal bases for the quantum superalgebra $U_q(\mathfrak{gl}(m, n))$* , J. Amer. Math. Soc. **13**, 295 (2000), doi:[10.1090/S0894-0347-00-00321-0](https://doi.org/10.1090/S0894-0347-00-00321-0).
- [14] A. A. Hagberg, D. A. Schult and P. J. Swart, *Exploring network structure, dynamics, and function using NetworkX*, In G. Varoquaux, T. Vaught and J. Millman, eds., *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pp. 11–15. Pasadena, CA, https://conference.scipy.org/proceedings/SciPy2008/paper_2/ (2008).
- [15] J. D. Hunter, *Matplotlib: a 2D graphics environment*, Comput. Sci. Eng. **9**, 90 (2007), doi:[10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [16] H. Krekel, B. Oliveira, R. Pfannschmidt, F. Bruynooghe, B. Laughner, F. Bruhin *et al.*, *pytest: simple powerful testing with Python*, <https://github.com/pytest-dev/pytest> (2004).
- [17] T. B. Terekli, *quantum-group: Exact SymPy verification of explicit $U_q(\mathfrak{sl}_2)$ and $GL_q(2|1)$ matrix identities*, Zenodo, doi:[10.5281/zenodo.23015576](https://doi.org/10.5281/zenodo.23015576), Version 1.1.0 (2026).